Numerical errors in computer simulations

Dr.ir. Ivo Roghair, Prof.dr.ir. Martin van Sint Annaland

Complex Fluids and Interfaces group
Eindhoven University of Technology

Numerical Methods (6BER03), 2025-2026

Today's outline

- Introduction
- Roundoff errors
- Break errors
- Loss of digits
- (Un)stable methods
  - Error propagation
- Symbolic math
- Summary

Today's outline

- Introduction
- Roundoff errors
- Break errors
- Loss of digits
- (Un)stable methods
  - Error propagation
- Symbolic math
- Summary

Example 1

Start your spreadsheet program (Excel, ...)

Example 1

Start your spreadsheet program (Excel, ...)

Enter:

Cell	Value
A1	`0.1`

Example 1

Start your spreadsheet program (Excel, ...)

Enter:

Cell	Value
A1	`0.1`
A2	`=(A1*10)-0.9`

Example 1

Start your spreadsheet program (Excel, ...)

Enter:

Cell	Value
A1	`0.1`
A2	`=(A1*10)-0.9`
A3	`=(A2*10)-0.9`

Example 1

Start your spreadsheet program (Excel, ...)

Enter:

Cell	Value
A1	`0.1`
A2	`=(A1*10)-0.9`
A3	`=(A2*10)-0.9`
A4	`=(A3*10)-0.9`

(repeat until A30)

Example 1

Start your spreadsheet program (Excel, ...)

Enter:

Cell	Value
A1	`0.1`
A2	`=(A1*10)-0.9`
A3	`=(A2*10)-0.9`
A4	`=(A3*10)-0.9`

(repeat until A30)

What's happening?

Example 1

Start your spreadsheet program (Excel, ...)

Enter:

Cell	Value
A1	`0.1`
A2	`=(A1*10)-0.9`
A3	`=(A2*10)-0.9`
A4	`=(A3*10)-0.9`

(repeat until A30)

What's happening?

Enter:

Cell	Value
A1	`2`

Example 1

Start your spreadsheet program (Excel, ...)

Enter:

Cell	Value
A1	`0.1`
A2	`=(A1*10)-0.9`
A3	`=(A2*10)-0.9`
A4	`=(A3*10)-0.9`

(repeat until A30)

What's happening?

Enter:

Cell	Value
A1	`2`
A2	`=(A1*10)-18`
A3	`=(A2*10)-18`
A4	`=(A3*10)-18`

(repeat until A30)

Example 2

Start Python

Example 2

Start Python

Investigate the result of `np.sin(1e40 * np.pi)`

Example 2

Start Python

Investigate the result of `np.sin(1e40 * np.pi)`

Create a vector `v` containing the powers of 10, e.g., from 10^0 up to 10^{40}, and solve `np.sin(v * np.pi)`:

```
import numpy as np
import matplotlib.pyplot as plt

v = np.logspace(0, 40, 41)
y = np.sin(v * np.pi)
plt.loglog(v, np.abs(y)) # Double log plot, values on y-axis must be positive.
plt.show()
```

Errors in computer simulations

In this lecture I will outline different numerical errors that can appear in computer simulations, and how these errors can affect the simulation results.

Errors in computer simulations

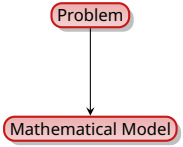
In this lecture I will outline different numerical errors that can appear in computer simulations, and how these errors can affect the simulation results.

- Errors in the mathematical model (physics)

Verification and validation

Problem

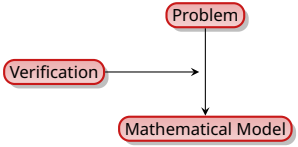
Verification and validation



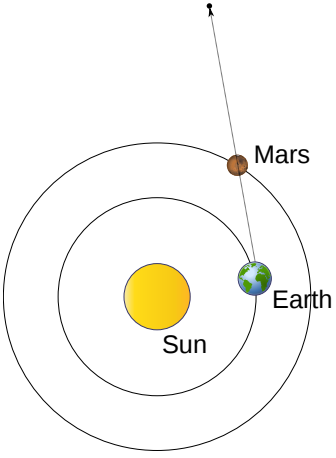
Verification and validation

Verification

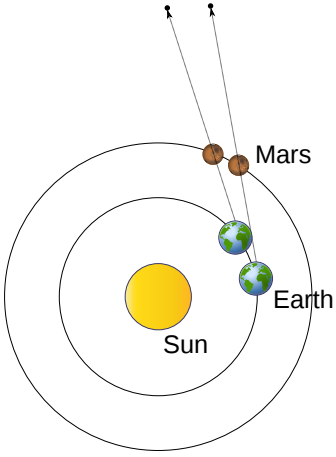
Verification is the process of mathematically and computationally assuring that the model computes the equations you intended to implement.



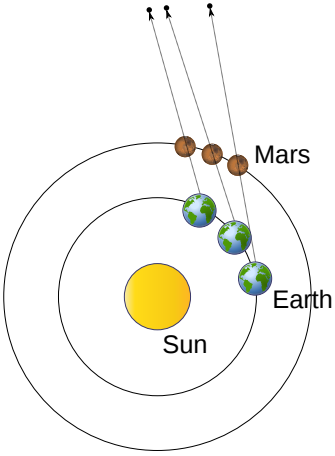
Verification of the physical model



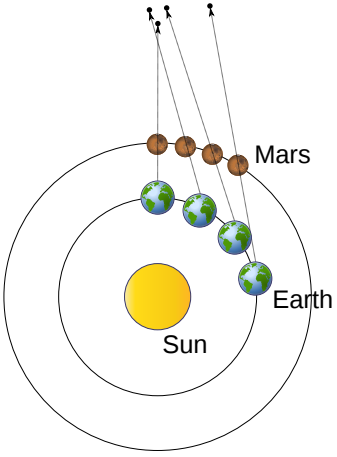
Verification of the physical model



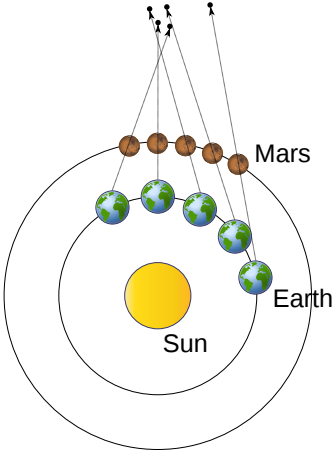
Verification of the physical model



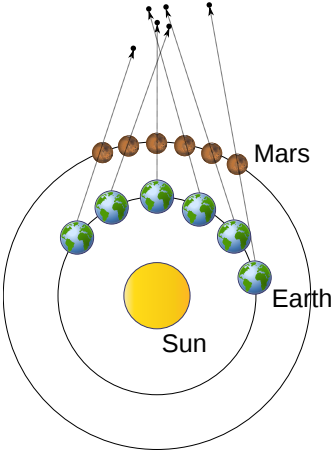
Verification of the physical model



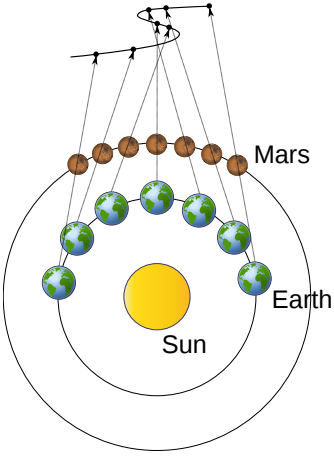
Verification of the physical model

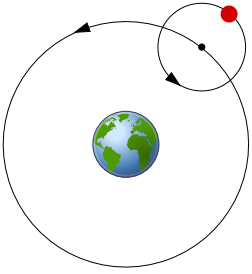


Verification of the physical model



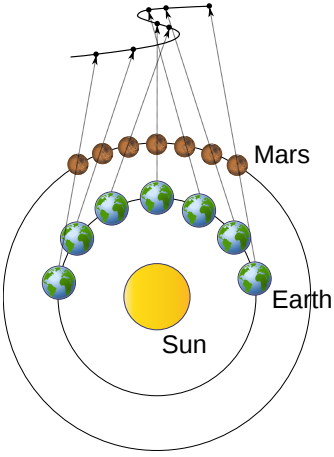
Verification of the physical model

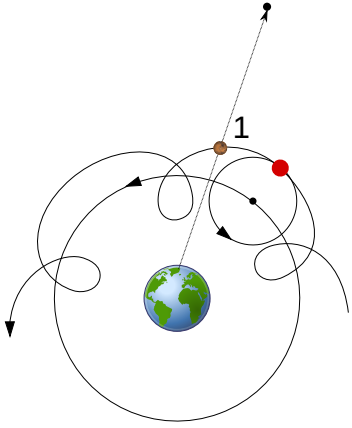




- The perceived orbit of Mars from Earth shows a zig-zag (in contrast to the Sun, Mercury, Venus)

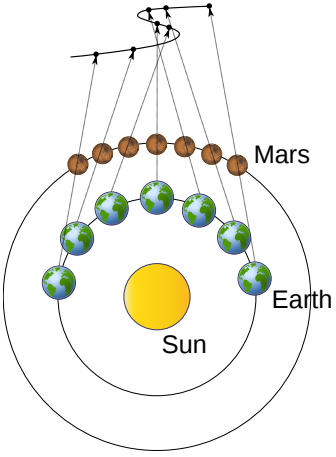
Verification of the physical model

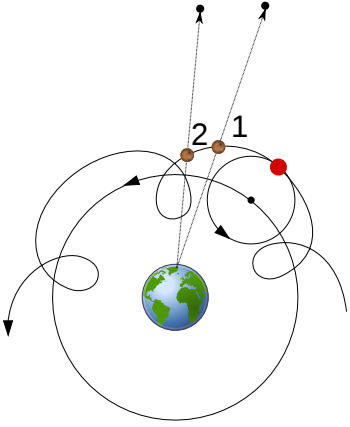




- The perceived orbit of Mars from Earth shows a zig-zag (in contrast to the Sun, Mercury, Venus)

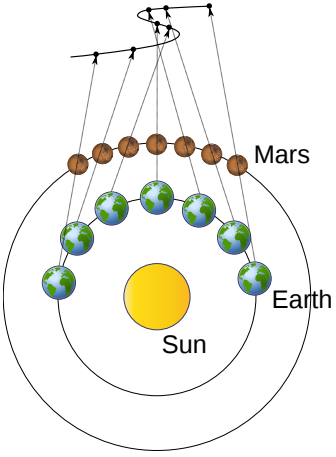
Verification of the physical model

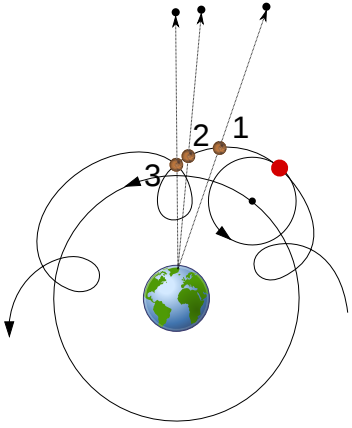




- The perceived orbit of Mars from Earth shows a zig-zag (in contrast to the Sun, Mercury, Venus)

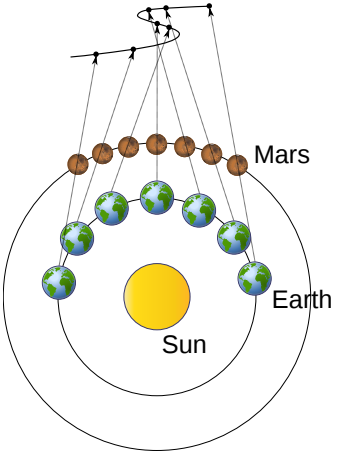
Verification of the physical model

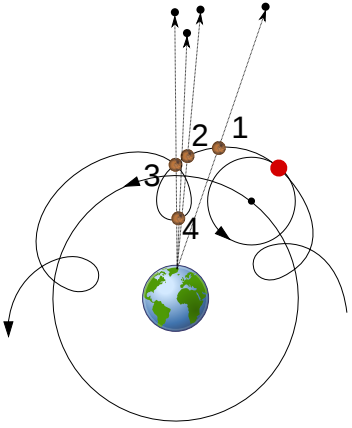




- The perceived orbit of Mars from Earth shows a zig-zag (in contrast to the Sun, Mercury, Venus)

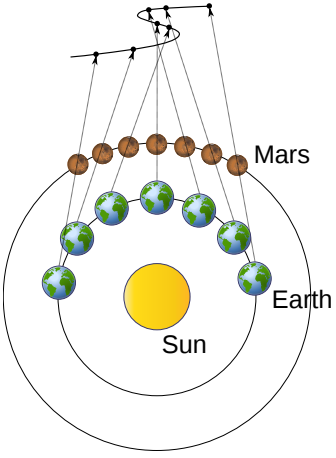
Verification of the physical model

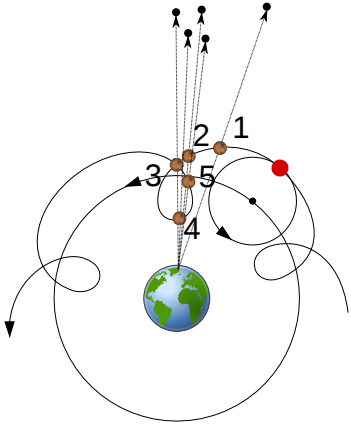




- The perceived orbit of Mars from Earth shows a zig-zag (in contrast to the Sun, Mercury, Venus)

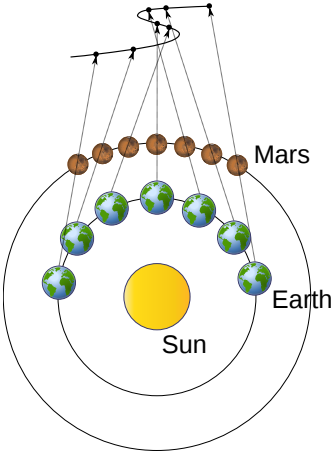
Verification of the physical model

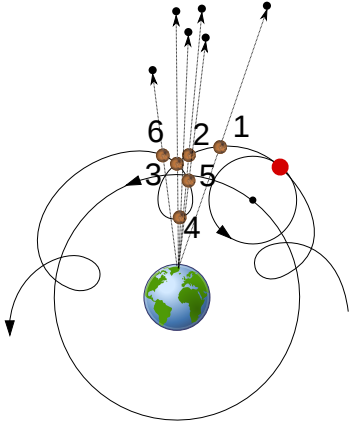




- The perceived orbit of Mars from Earth shows a zig-zag (in contrast to the Sun, Mercury, Venus)

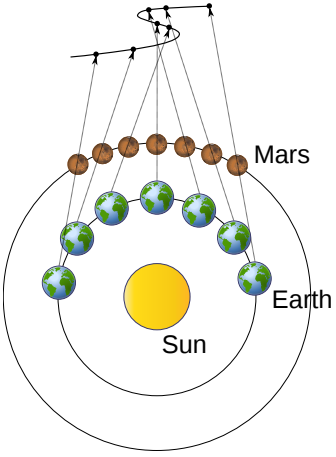
Verification of the physical model

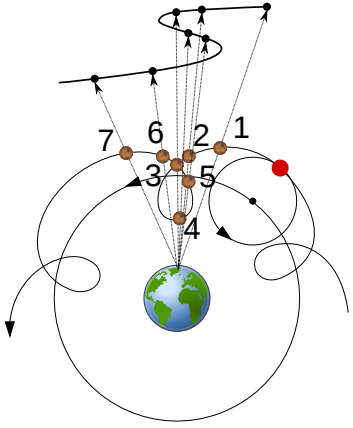




- The perceived orbit of Mars from Earth shows a zig-zag (in contrast to the Sun, Mercury, Venus)

Verification of the physical model





- The perceived orbit of Mars from Earth shows a zig-zag (in contrast to the Sun, Mercury, Venus)
- Even though they were not 'right', Earth-centered models (Ptolemy) were still valid

Be aware of your uncertainties

Aleatory uncertainty

Uncertainty that arises due to inherent randomness of the system, features that are too complex to measure and take into account

Epistemic uncertainty

Uncertainty that arises due to lack of knowledge of the system, but could in principle be known

Errors in computer simulations

In this lecture I will outline different numerical errors that can appear in computer simulations, and how these errors can affect the simulation results.

- Errors in the mathematical model (physics)

Errors in computer simulations

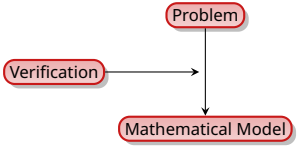
In this lecture I will outline different numerical errors that can appear in computer simulations, and how these errors can affect the simulation results.

- Errors in the mathematical model (physics)
- Errors in the program (implementation)

Verification and validation

Verification

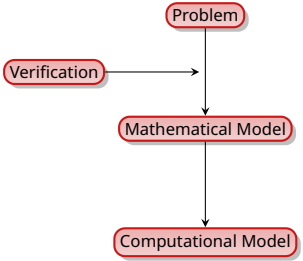
Verification is the process of mathematically and computationally assuring that the model computes the equations you intended to implement.



Verification and validation

Verification

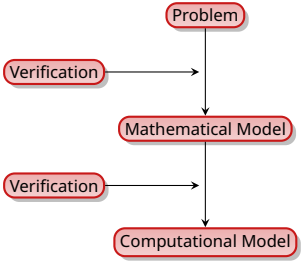
Verification is the process of mathematically and computationally assuring that the model computes the equations you intended to implement.



Verification and validation

Verification

Verification is the process of mathematically and computationally assuring that the model computes the equations you intended to implement.



Errors in computer simulations

In this lecture I will outline different numerical errors that can appear in computer simulations, and how these errors can affect the simulation results.

- Errors in the mathematical model (physics)
- Errors in the program (implementation)

Errors in computer simulations

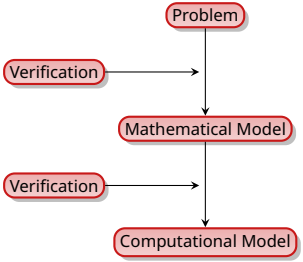
In this lecture I will outline different numerical errors that can appear in computer simulations, and how these errors can affect the simulation results.

- Errors in the mathematical model (physics)
- Errors in the program (implementation)
- Errors in the entered parameters

Verification and validation

Verification

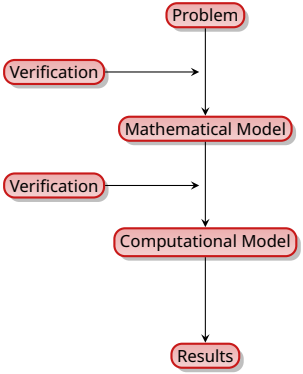
Verification is the process of mathematically and computationally assuring that the model computes the equations you intended to implement.



Verification and validation

Verification

Verification is the process of mathematically and computationally assuring that the model computes the equations you intended to implement.



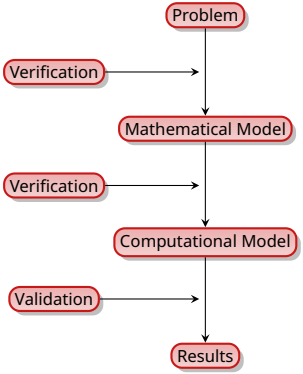
Verification and validation

Verification

Verification is the process of mathematically and computationally assuring that the model computes the equations you intended to implement.

Validation

Validation is the process of determining the degree to which a model is an accurate representation of the real world from the perspective of the intended uses of the model



Errors in computer simulations

In this lecture I will outline different numerical errors that can appear in computer simulations, and how these errors can affect the simulation results.

- Errors in the mathematical model (physics)
- Errors in the program (implementation)
- Errors in the entered parameters

Errors in computer simulations

In this lecture I will outline different numerical errors that can appear in computer simulations, and how these errors can affect the simulation results.

- Errors in the mathematical model (physics)
- Errors in the program (implementation)
- Errors in the entered parameters
- Roundoff- and truncation errors

Errors in computer simulations

In this lecture I will outline different numerical errors that can appear in computer simulations, and how these errors can affect the simulation results.

- Errors in the mathematical model (physics)
- Errors in the program (implementation)
- Errors in the entered parameters
- Roundoff- and truncation errors
- Break errors

Errors in computer simulations

In this lecture I will outline different numerical errors that can appear in computer simulations, and how these errors can affect the simulation results.

- Errors in the mathematical model (physics)
- Errors in the program (implementation)
- Errors in the entered parameters
- Roundoff- and truncation errors
- Break errors

Significant digits

A numerical result $\tilde{x}$ is an approximation of the real value x.

- Absolute error

$$\delta = |\tilde{x} - x|, x \neq 0$$

- Relative error

$$\frac{\delta}{\tilde{x}} = |\frac{\tilde{x} - x}{\tilde{x}}|$$

- Error margin

$$\tilde{x} - \delta \leq x \leq \tilde{x} + \delta$$

$$x = \tilde{x} \pm \delta$$

Significant digits

A numerical result $\tilde{x}$ is an approximation of the real value x.

- Absolute error

$$\delta = |\tilde{x} - x|, x \neq 0$$

- Relative error

$$\frac{\delta}{\tilde{x}} = |\frac{\tilde{x} - x}{\tilde{x}}|$$

- Error margin

$$\tilde{x} - \delta \leq x \leq \tilde{x} + \delta$$

$$x = \tilde{x} \pm \delta$$

Significant digits

A numerical result $\tilde{x}$ is an approximation of the real value x.

- Absolute error

$$\delta = |\tilde{x} - x|, x \neq 0$$

- Relative error

$$\frac{\delta}{\tilde{x}} = |\frac{\tilde{x} - x}{\tilde{x}}|$$

- Error margin

$$\tilde{x} - \delta \leq x \leq \tilde{x} + \delta$$

$$x = \tilde{x} \pm \delta$$

Significant digits

- $\tilde{x}$ has m significant digits if the absolute error in x is smaller or equal to 5 at the $(m+1)$-th position:

$$10^{q-1} \leq |\tilde{x}| \leq 10^{q}$$

$$|x - \tilde{x}| = 0.5 \times 10^{q-m}$$

- For example:

$$x = \frac{1}{3}, \tilde{x} = 0.333 \Rightarrow \delta = 0.00033333\ldots$$

3 significant digits

Significant digits

- $\tilde{x}$ has m significant digits if the absolute error in x is smaller or equal to 5 at the $(m+1)$-th position:

$$10^{q-1} \leq |\tilde{x}| \leq 10^{q}$$

$$|x - \tilde{x}| = 0.5 \times 10^{q-m}$$

- For example:

$$x = \frac{1}{3}, \tilde{x} = 0.333 \Rightarrow \delta = 0.00033333\ldots$$

3 significant digits

Today's outline

- Introduction
- Roundoff errors
- Break errors
- Loss of digits
- (Un)stable methods
  - Error propagation
- Symbolic math
- Summary

Representation of numbers

- Computers represent a number with a finite number of digits: each number is therefore an approximation due to roundoff and truncation errors.

Representation of numbers

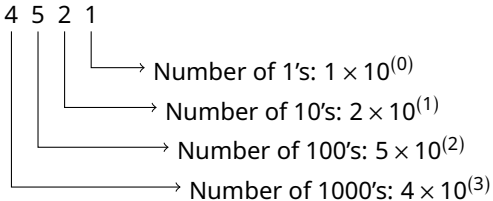
- Computers represent a number with a finite number of digits: each number is therefore an approximation due to roundoff and truncation errors.
- In the decimal system, a digit c at position n has a value of $c \times 10^{n-1}$

Representation of numbers

- Computers represent a number with a finite number of digits: each number is therefore an approximation due to roundoff and truncation errors.
- In the decimal system, a digit c at position n has a value of $c \times 10^{n-1}$

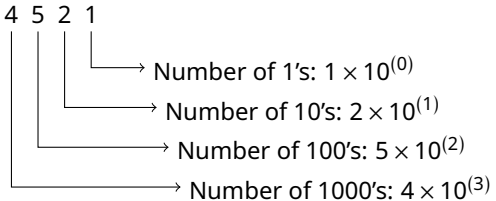
Representation of numbers

- Computers represent a number with a finite number of digits: each number is therefore an approximation due to roundoff and truncation errors.
- In the decimal system, a digit c at position n has a value of $c \times 10^{n-1}$



Representation of numbers

- Computers represent a number with a finite number of digits: each number is therefore an approximation due to roundoff and truncation errors.
- In the decimal system, a digit c at position n has a value of $c \times 10^{n-1}$



$$(4521)_{10} = 4 \times 10^3 + 5 \times 10^2 + 2 \times 10^1 + 1 \times 10^0$$

Representation of numbers

- You could use another basis, computers often use the basis 2:

$(4521)_{10} =$

$=($

Representation of numbers

- You could use another basis, computers often use the basis 2:

$$(4521)_{10} = 1 \times 2^{12} +$$

$$= (1$$

Representation of numbers

- You could use another basis, computers often use the basis 2:

$$(4521)_{10} = 1 \times 2^{12} + 0 \times 2^{11} +$$

$$=(10$$

Representation of numbers

- You could use another basis, computers often use the basis 2:

$$(4521)_{10} = 1 \times 2^{12} + 0 \times 2^{11} + 0 \times 2^{10} +$$

$$=(100$$

Representation of numbers

- You could use another basis, computers often use the basis 2:

$$(4521)_{10} = 1 \times 2^{12} + 0 \times 2^{11} + 0 \times 2^{10} + 0 \times 2^{9} +$$

$$=(1000$$

Representation of numbers

- You could use another basis, computers often use the basis 2:

$$(4521)_{10} = 1 \times 2^{12} + 0 \times 2^{11} + 0 \times 2^{10} + 0 \times 2^{9} + 1 \times 2^{8} + \ldots$$

$$=(10001$$

Representation of numbers

- You could use another basis, computers often use the basis 2:

$$(4521)_{10} = 1 \times 2^{12} + 0 \times 2^{11} + 0 \times 2^{10} + 0 \times 2^{9} + 1 \times 2^{8} + \ldots$$
$$\ldots 1 \times 2^{7} +$$

$$=(100011$$

Representation of numbers

- You could use another basis, computers often use the basis 2:

$$(4521)_{10} = 1 \times 2^{12} + 0 \times 2^{11} + 0 \times 2^{10} + 0 \times 2^{9} + 1 \times 2^{8} + \ldots$$
$$\ldots 1 \times 2^{7} + 0 \times 2^{6} +$$

$$= (1000110$$

Representation of numbers

- You could use another basis, computers often use the basis 2:

$$(4521)_{10} = 1 \times 2^{12} + 0 \times 2^{11} + 0 \times 2^{10} + 0 \times 2^{9} + 1 \times 2^{8} + \ldots$$
$$\ldots 1 \times 2^{7} + 0 \times 2^{6} + 1 \times 2^{5} +$$

$$= (10001101$$

Representation of numbers

- You could use another basis, computers often use the basis 2:

$$(4521)_{10} = 1 \times 2^{12} + 0 \times 2^{11} + 0 \times 2^{10} + 0 \times 2^{9} + 1 \times 2^{8} + \ldots$$
$$\ldots 1 \times 2^{7} + 0 \times 2^{6} + 1 \times 2^{5} + 0 \times 2^{4} + \ldots$$

$$= (100011010$$

Representation of numbers

- You could use another basis, computers often use the basis 2:

$$(4521)_{10} = 1 \times 2^{12} + 0 \times 2^{11} + 0 \times 2^{10} + 0 \times 2^{9} + 1 \times 2^{8} + \ldots$$
$$\ldots 1 \times 2^{7} + 0 \times 2^{6} + 1 \times 2^{5} + 0 \times 2^{4} + \ldots$$
$$\ldots 1 \times 2^{3} +$$
$$=(1000110101$$

Representation of numbers

- You could use another basis, computers often use the basis 2:

$$(4521)_{10} = 1 \times 2^{12} + 0 \times 2^{11} + 0 \times 2^{10} + 0 \times 2^{9} + 1 \times 2^{8} + \ldots$$
$$\ldots 1 \times 2^{7} + 0 \times 2^{6} + 1 \times 2^{5} + 0 \times 2^{4} + \ldots$$
$$\ldots 1 \times 2^{3} + 0 \times 2^{2} +$$
$$= (10001101010$$

Representation of numbers

- You could use another basis, computers often use the basis 2:

$$(4521)_{10} = 1 \times 2^{12} + 0 \times 2^{11} + 0 \times 2^{10} + 0 \times 2^{9} + 1 \times 2^{8} + \ldots$$
$$\ldots 1 \times 2^{7} + 0 \times 2^{6} + 1 \times 2^{5} + 0 \times 2^{4} + \ldots$$
$$\ldots 1 \times 2^{3} + 0 \times 2^{2} + 0 \times 2^{1} +$$
$$= (100011010100$$

Representation of numbers

- You could use another basis, computers often use the basis 2:

$$
\begin{aligned}
(4521)_{10} &= 1\times 2^{12} + 0\times 2^{11} + 0\times 2^{10} + 0\times 2^{9} + 1\times 2^{8} + \dots \\
&\dots 1\times 2^{7} + 0\times 2^{6} + 1\times 2^{5} + 0\times 2^{4} + \dots \\
&\dots 1\times 2^{3} + 0\times 2^{2} + 0\times 2^{1} + 1\times 2^{0} \\
&= (1000110101001)_2
\end{aligned}
$$

Representation of numbers

- You could use another basis, computers often use the basis 2:

$$
\begin{aligned}
(4521)_{10} &= 1\times 2^{12} + 0\times 2^{11} + 0\times 2^{10} + 0\times 2^{9} + 1\times 2^{8} + \dots \\
&\dots 1\times 2^{7} + 0\times 2^{6} + 1\times 2^{5} + 0\times 2^{4} + \dots \\
&\dots 1\times 2^{3} + 0\times 2^{2} + 0\times 2^{1} + 1\times 2^{0} \\
&= (1000110101001)_2
\end{aligned}
$$

- In general:

$$(c_m \dots c_1 c_0)_q = c_0 q^0 + c_1 q^1 + \dots + c_m q^m, c \in \{0, 1, 2, \dots, q-1\}$$

Representation of numbers

- Numbers are stored in binary in the memory of a computer, in segments of a specific length (called a *word*).
- We distinguish multiple types of numbers:
  - Integers: $-301, -1, 0, 1, 96, 2293, \ldots$
  - Floating points: $-301.01, 0.01, 3.14159265, 14498.2$
- A binary integer representation looks like the following bit sequence:

$$z = \sigma\left(c_0 2^0 + c_1 2^1 + \ldots + c_{\lambda-1} 2^{\lambda-1}\right)$$

  σ is the sign of z (+ or -), and λ is the length of the word
- Endianness: the order of bits stored by a computer

Representation of numbers

- Numbers are stored in binary in the memory of a computer, in segments of a specific length (called a *word*).
- We distinguish multiple types of numbers:
  - Integers: $-301, -1, 0, 1, 96, 2293, \ldots$
  - Floating points: $-301.01, 0.01, 3.14159265, 14498.2$
- A binary integer representation looks like the following bit sequence:

$$z = \sigma\left(c_0 2^0 + c_1 2^1 + \ldots + c_{\lambda-1} 2^{\lambda-1}\right)$$

  σ is the sign of z (+ or -), and λ is the length of the word
- Endianness: the order of bits stored by a computer

Representation of numbers

- Numbers are stored in binary in the memory of a computer, in segments of a specific length (called a *word*).
- We distinguish multiple types of numbers:
  - Integers: $-301, -1, 0, 1, 96, 2293, \ldots$
  - Floating points: $-301.01, 0.01, 3.14159265, 14498.2$
- A binary integer representation looks like the following bit sequence:

$$z = \sigma\left(c_0 2^0 + c_1 2^1 + \ldots + c_{\lambda-1} 2^{\lambda-1}\right)$$

  σ is the sign of z (+ or -), and λ is the length of the word
- Endianness: the order of bits stored by a computer

Representation of numbers

- Numbers are stored in binary in the memory of a computer, in segments of a specific length (called a *word*).
- We distinguish multiple types of numbers:
  - Integers: $-301, -1, 0, 1, 96, 2293, \ldots$
  - Floating points: $-301.01, 0.01, 3.14159265, 14498.2$
- A binary integer representation looks like the following bit sequence:

$$z = \sigma\left(c_0 2^0 + c_1 2^1 + \ldots + c_{\lambda-1} 2^{\lambda-1}\right)$$

  σ is the sign of z (+ or -), and λ is the length of the word
- Endianness: the order of bits stored by a computer

Excercise

- Convert the following decimal number to base-2: 214

Excercise

- Convert the following decimal number to base-2: 214

$$214_{10} = 11010110_2$$

Excercise

- Convert the following decimal number to base-2: 214

$$214_{10} = 11010110_2$$

- Excel:
  - Decimal: `=DEC2BIN(214)`
  - Octal: `=DEC2OCT(214)`
  - Hexadecimal: `=DEC2HEX(214)`

Excercise

- Convert the following decimal number to base-2: 214

$$214_{10} = 11010110_2$$

- Excel:
  - Decimal: `=DEC2BIN(214)`
  - Octal: `=DEC2OCT(214)`
  - Hexadecimal: `=DEC2HEX(214)`
- Python:
  - Decimal: `bin(214)`
  - Octal: `oct(214)`
  - Hexadecimal: `hex(214)`

Try yourself

Try the following commands in Python:

Command	Result
`np.iinfo(np.int32).min`	`-2147483648`

Try yourself

Try the following commands in Python:

Command	Result
`np.iinfo(np.int32).min`	`-2147483648`
`np.iinfo(np.int32).max`	`2147483647`

Try yourself

Try the following commands in Python:

Command	Result
`np.iinfo(np.int32).min`	`-2147483648`
`np.iinfo(np.int32).max`	`2147483647`
`i = np.int16(np.iinfo(np.int32).max)`	`i = 32767`

Try yourself

Try the following commands in Python:

Command	Result
`np.iinfo(np.int32).min`	`-2147483648`
`np.iinfo(np.int32).max`	`2147483647`
`i = np.int16(np.iinfo(np.int32).max)`	`i = 32767`
`type(i)`	`<class 'numpy.int16'>`

Try yourself

Try the following commands in Python:

Command	Result
`np.iinfo(np.int32).min`	`-2147483648`
`np.iinfo(np.int32).max`	`2147483647`
`i = np.int16(np.iinfo(np.int32).max)`	`i = 32767`
`type(i)`	`<class 'numpy.int16'>`
`i = i + 100`	`i = 32767`

Try yourself

Try the following commands in Python:

Command	Result
`np.iinfo(np.int32).min`	`-2147483648`
`np.iinfo(np.int32).max`	`2147483647`
`i = np.int16(np.iinfo(np.int32).max)`	`i = 32767`
`type(i)`	`<class 'numpy.int16'>`
`i = i + 100`	`i = 32767`
`np.finfo(np.float64).max`	`1.7976931348623157e+308`

Try yourself

Try the following commands in Python:

Command	Result
`np.iinfo(np.int32).min`	`-2147483648`
`np.iinfo(np.int32).max`	`2147483647`
`i = np.int16(np.iinfo(np.int32).max)`	`i = 32767`
`type(i)`	`<class 'numpy.int16'>`
`i = i + 100`	`i = 32767`
`np.finfo(np.float64).max`	`1.7976931348623157e+308`
`f = 0.1`	

Try yourself

Try the following commands in Python:

Command	Result
`np.iinfo(np.int32).min`	`-2147483648`
`np.iinfo(np.int32).max`	`2147483647`
`i = np.int16(np.iinfo(np.int32).max)`	`i = 32767`
`type(i)`	`<class 'numpy.int16'>`
`i = i + 100`	`i = 32767`
`np.finfo(np.float64).max`	`1.7976931348623157e+308`
`f = 0.1`	
`type(f)`	`<class 'float'>`

Try yourself

Try the following commands in Python:

Command	Result
`np.iinfo(np.int32).min`	`-2147483648`
`np.iinfo(np.int32).max`	`2147483647`
`i = np.int16(np.iinfo(np.int32).max)`	`i = 32767`
`type(i)`	`<class 'numpy.int16'>`
`i = i + 100`	`i = 32767`
`np.finfo(np.float64).max`	`1.7976931348623157e+308`
`f = 0.1`	
`type(f)`	`<class 'float'>`
`print(":.16f".format(f))`	`0.1000000000000000`

Try yourself

Try the following commands in Python:

Command	Result
`np.iinfo(np.int32).min`	`-2147483648`
`np.iinfo(np.int32).max`	`2147483647`
`i = np.int16(np.iinfo(np.int32).max)`	`i = 32767`
`type(i)`	`<class 'numpy.int16'>`
`i = i + 100`	`i = 32767`
`np.finfo(np.float64).max`	`1.7976931348623157e+308`
`f = 0.1`	
`type(f)`	`<class 'float'>`
`print(":.16f".format(f))`	`0.1000000000000000`
`print(":.20f".format(f))`	`0.10000000000000000555`

Representation of integer numbers

- In Python, integers are typically represented with at least 32 bits (though this can be larger depending on the Python implementation and system architecture). For a typical 64-bit Python installation, the integer type would have 64 bits.

Representation of integer numbers

- In Python, integers are typically represented with at least 32 bits (though this can be larger depending on the Python implementation and system architecture). For a typical 64-bit Python installation, the integer type would have 64 bits.
- The set of numbers that a 32-bit integer 'int32' can represent in numpy is:

$$-2^{31} \leq z \leq 2^{31} - 1 \approx 2 \times 10^9$$

where z is a variable representing an `np.int32` type value.

Representation of integer numbers

- In Python, integers are typically represented with at least 32 bits (though this can be larger depending on the Python implementation and system architecture). For a typical 64-bit Python installation, the integer type would have 64 bits.
- The set of numbers that a 32-bit integer ‘int32‘ can represent in numpy is:

$$-2^{31} \leq z \leq 2^{31} - 1 \approx 2 \times 10^9$$

  where z is a variable representing an `np.int32` type value.
- If, during a calculation, an integer number in Python exceeds its maximum limit, Python automatically promotes it to a larger type if possible. In the case of numpy, an overflow error will be thrown if a number exceeds the maximum value representable by the ‘np.int32‘ type.

Representation of integer numbers

- In Python, integers are typically represented with at least 32 bits (though this can be larger depending on the Python implementation and system architecture). For a typical 64-bit Python installation, the integer type would have 64 bits.
- The set of numbers that a 32-bit integer 'int32' can represent in numpy is:

$$-2^{31} \leq z \leq 2^{31} - 1 \approx 2 \times 10^9$$

  where z is a variable representing an `np.int32` type value.
- If, during a calculation, an integer number in Python exceeds its maximum limit, Python automatically promotes it to a larger type if possible. In the case of numpy, an overflow error will be thrown if a number exceeds the maximum value representable by the 'np.int32' type.
- How can a computer identify an overflow? Usually through error reporting mechanisms within the language. Python's numpy library, for instance, will throw an OverflowError when an operation results in a number larger than what can be represented by the current data type.

Representation of real (floating point) numbers

- Formally, a real number is represented by the following bit sequence

$$x = \sigma\left(2^{-1} + c_2 2^{-2} + \ldots + c_m 2^{-m}\right) 2^{e-1023}$$

  Here, σ is the sign of x and e is an integer value.

- A floating point number hence contains sections that contain the sign, the exponent and the mantissa

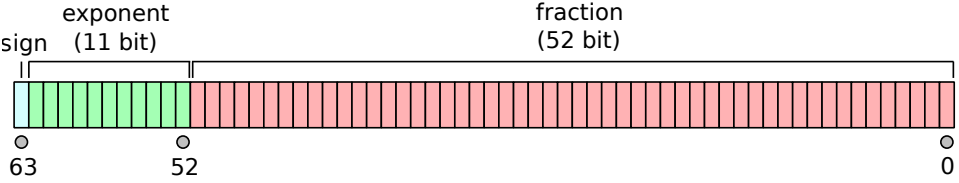


Image: Wikimedia Commons CC by-SA

Roundoff error: truncation vs rounding

Real numbers are stored with a limited number of digits (or bits). When a number does not fit, the representation must be *approximated*.

Truncation (chopping)

- Extra digits are simply discarded.
- Always biases the result in the same direction.

$$\pi = 3.14159265 \implies 3.14$$

Rounding (to nearest)

- Last retained digit is rounded up or down.
- Error is smaller and approximately unbiased.

$$\pi = 3.14159265 \implies 3.14, \qquad 3.145 \Rightarrow 3.15$$

In binary floating-point, this corresponds to choosing the last bit as 0 or 1 when the exact value lies in between.

Truncation vs rounding: numerical example

Simulate decimal arithmetic with 3 significant digits.

```python
import math

x = math.pi

# chopping to 3 significant digits
chop = float(f"{x:.3g}")

# rounding to nearest (default Python behavior)
rounding = round(x, 3)

print(f"'exact' : {x}...")
print(f"chopped : {chop}")
print(f"rounded : {rounding}")
print(f"errors : {chop-x}, {rounding-x}")
```

```
'exact' : 3.141592653589793...
chopped : 3.14
rounded : 3.142
errors : -0.0015926535897929917,
  0.0004073464102067881
```

Truncation vs rounding: numerical example

Simulate decimal arithmetic with 3 significant digits.

```python
import math

x = math.pi

# chopping to 3 significant digits
chop = float(f"{x:.3g}")

# rounding to nearest (default Python behavior)
rounding = round(x, 3)

print(f"'exact' : {x}...")
print(f"chopped : {chop}")
print(f"rounded : {rounding}")
print(f"errors : {chop-x}, {rounding-x}")
```

```
'exact' : 3.141592653589793...
chopped : 3.14
rounded : 3.142
errors : -0.0015926535897929917,
  0.0004073464102067881
```

- Chopping always underestimates here.
- Rounding alternates between over- and underestimation.

Exercise: binary representation

Consider a binary floating point number, here represented as $x = 1.101101011_2$.

- The digits before the binary point represent powers of $2^0, 2^1, \ldots$ (from right to left)
- The digits after the binary point represent powers of $2^{-1}, 2^{-2}, \ldots$

Write a function to convert a string representation of a binary number (e.g. `'1.101101011'`) to its decimal value.

Exercise: binary representation

Consider a binary floating point number, here represented as $x = 1.101101011_2$.

- The digits before the binary point represent powers of $2^0, 2^1, \ldots$ (from right to left)
- The digits after the binary point represent powers of $2^{-1}, 2^{-2}, \ldots$

Write a function to convert a string representation of a binary number (e.g. `'1.101101011'`) to its decimal value.

```
def binary_to_float(_bin: str) -> float:
    _pos,_neg = _bin.split('.')
    _float = 0
    # Count positive powers
    for i in range(len(_pos)):
        _float += int(_pos[i]) * 2**i
    # Count negative powers
    for i in range(1,len(_neg)+1):
        _float += int(_neg[i-1]) * 2**(-i)
    return _float

my_bin = '1.101'
print(binary_to_float(my_bin))
```

Binary truncation (chopping)

Example: store $x = 1.101101011_2$ using only $p = 5$ bits.

$$x = 1.\underline{10110}1011_2 \implies \mathrm{fl}_{\mathrm{chop}}(x) = 1.10110_2$$

Properties:

- Always rounds *towards zero*.
- Error has fixed sign.
- Bias accumulates in long computations.

Relative error:

$$\frac{|x - \mathrm{fl}(x)|}{|x|} < 2^{-p}.$$

Binary rounding (to nearest)

Same number, same precision ($p = 5$ bits):

$$x = 1.\underline{10110}1011_2$$

Look at the first discarded bit:

$$\mathrm{fl}_{\text{round}}(x) = 1.10111_2$$

Properties:

- Error is smaller than with chopping.
- Over- and underestimation occur with similar probability.
- Expected rounding error is approximately zero.

Bound on relative error:

$$\frac{|x - \mathrm{fl}(x)|}{|x|} \leq \tfrac{1}{2}2^{-p}.$$

Today's outline

- Introduction
- Roundoff errors
- Break errors
- Loss of digits
- (Un)stable methods
  - Error propagation
- Symbolic math
- Summary

Trigonometric, Logarithmic, and Exponential computations

- Processors can do logic and arithmetic instructions
- Trigonometric, logarithmic and exponential calculations are “higher-level” functions: exp, sin, cos, tan, sec, arcsin, arccos, arctan, log, ln, …
- Such functions can be performed using these “low level” instructions, for instance using a Taylor series:

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{x!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots$$

Trigonometric, Logarithmic, and Exponential computations

- Processors can do logic and arithmetic instructions
- Trigonometric, logarithmic and exponential calculations are “higher-level” functions: exp, sin, cos, tan, sec, arcsin, arccos, arctan, log, ln, …
- Such functions can be performed using these “low level” instructions, for instance using a Taylor series:

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{x!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots$$

Trigonometric, Logarithmic, and Exponential computations

- Processors can do logic and arithmetic instructions
- Trigonometric, logarithmic and exponential calculations are “higher-level” functions: exp, sin, cos, tan, sec, arcsin, arccos, arctan, log, ln, …
- Such functions can be performed using these “low level” instructions, for instance using a Taylor series:

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{x!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots$$

Trigonometric, Logarithmic, and Exponential computations

- These operations involve many multiplications and additions, and are therefore *expensive*
- Computations can only take finite time, for infinite series, calculations are interrupted at N

$$\sin(x) = \sum_{n=0}^{N} \frac{(-1)^n}{(2n+1)!} x^{2n+1} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \ldots + \frac{(-1)^N}{(2N+1)!} x^{2N+1}$$

$$e^x = \sum_{n=0}^{N} \frac{x^n}{x!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \ldots + \frac{x^N}{N!}$$

- This results in a *break error*

Trigonometric, Logarithmic, and Exponential computations

- These operations involve many multiplications and additions, and are therefore *expensive*
- Computations can only take finite time, for infinite series, calculations are interrupted at N

$$\sin(x) = \sum_{n=0}^{N} \frac{(-1)^n}{(2n+1)!} x^{2n+1} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \ldots + \frac{(-1)^N}{(2N+1)!} x^{2N+1}$$

$$e^x = \sum_{n=0}^{N} \frac{x^n}{x!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \ldots + \frac{x^N}{N!}$$

- This results in a *break error*

Trigonometric, Logarithmic, and Exponential computations

- These operations involve many multiplications and additions, and are therefore *expensive*
- Computations can only take finite time, for infinite series, calculations are interrupted at N

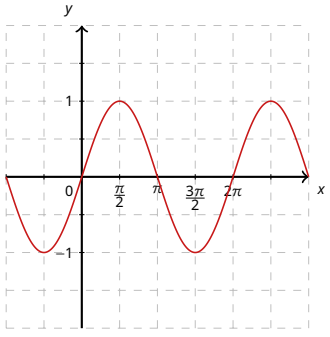
$$\sin(x) = \sum_{n=0}^{N} \frac{(-1)^n}{(2n+1)!} x^{2n+1} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \ldots + \frac{(-1)^N}{(2N+1)!} x^{2N+1}$$

$$e^x = \sum_{n=0}^{N} \frac{x^n}{x!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \ldots + \frac{x^N}{N!}$$

- This results in a *break error*

Algorithm for sine-computation

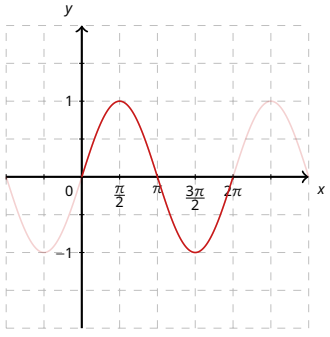
A computer may use a clever algorithm to limit the number of operations required to perform a higher-level function. A (fictional!) example for the computation of $\sin(x)$:



Algorithm for sine-computation

A computer may use a clever algorithm to limit the number of operations required to perform a higher-level function. A (fictional!) example for the computation of $\sin(x)$:

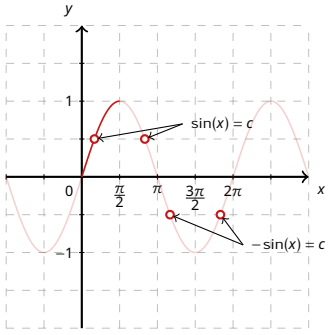
1. Use periodicity so that $0 \leq x \leq 2\pi$



Algorithm for sine-computation

A computer may use a clever algorithm to limit the number of operations required to perform a higher-level function. A (fictional!) example for the computation of $\sin(x)$:

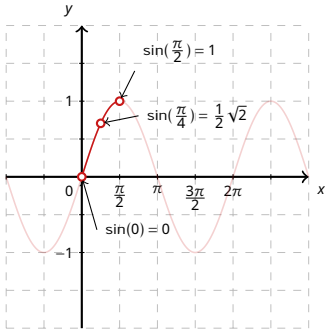
1. Use periodicity so that $0 \leq x \leq 2\pi$
2. Use symmetry ($0 \leq x \leq \frac{\pi}{2}$)



Algorithm for sine-computation

A computer may use a clever algorithm to limit the number of operations required to perform a higher-level function. A (fictional!) example for the computation of $\sin(x)$:

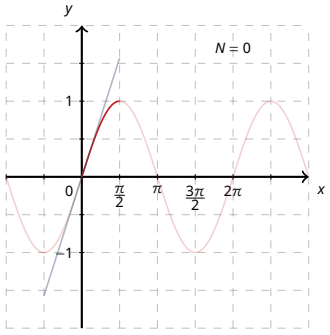
1. Use periodicity so that $0 \leq x \leq 2\pi$
2. Use symmetry ($0 \leq x \leq \frac{\pi}{2}$)
3. Use lookup tables for known values



Algorithm for sine-computation

A computer may use a clever algorithm to limit the number of operations required to perform a higher-level function. A (fictional!) example for the computation of $\sin(x)$:

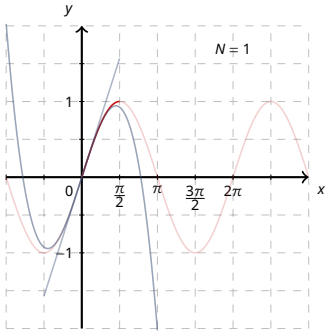
1. Use periodicity so that $0 \leq x \leq 2\pi$
2. Use symmetry ($0 \leq x \leq \frac{\pi}{2}$)
3. Use lookup tables for known values
4. Perform taylor expansion



Algorithm for sine-computation

A computer may use a clever algorithm to limit the number of operations required to perform a higher-level function. A (fictional!) example for the computation of $\sin(x)$:

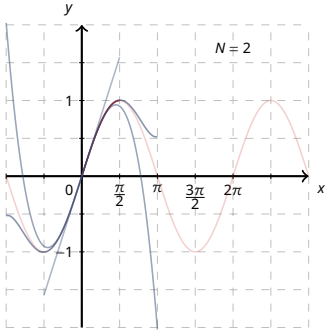
1. Use periodicity so that $0 \leq x \leq 2\pi$
2. Use symmetry ($0 \leq x \leq \frac{\pi}{2}$)
3. Use lookup tables for known values
4. Perform taylor expansion



Algorithm for sine-computation

A computer may use a clever algorithm to limit the number of operations required to perform a higher-level function. A (fictional!) example for the computation of $\sin(x)$:

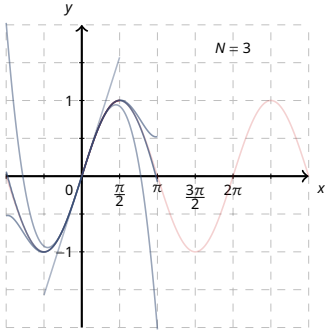
1. Use periodicity so that $0 \leq x \leq 2\pi$
2. Use symmetry ($0 \leq x \leq \frac{\pi}{2}$)
3. Use lookup tables for known values
4. Perform taylor expansion



Algorithm for sine-computation

A computer may use a clever algorithm to limit the number of operations required to perform a higher-level function. A (fictional!) example for the computation of $\sin(x)$:

1. Use periodicity so that $0 \leq x \leq 2\pi$
2. Use symmetry ($0 \leq x \leq \frac{\pi}{2}$)
3. Use lookup tables for known values
4. Perform taylor expansion



Today's outline

- Introduction
- Roundoff errors
- Break errors
- Loss of digits
- (Un)stable methods
  - Error propagation
- Symbolic math
- Summary

Arithmetic operations with binary numbers

Addition:

$0 + 0 = 0$
$0 + 1 = 1$
$1 + 0 = 1$
$1 + 1 = 0$ (carry one)

Arithmetic operations with binary numbers

$0 + 0 = 0$
$0 + 1 = 1$
$1 + 0 = 1$
$1 + 1 = 0$ (carry one)

Addition:

$$\begin{array}{rccc} & 1 & 4 & 5 \\ + & & 2 & 3 \\ \hline & 1 & 6 & 8 \end{array}$$

Arithmetic operations with binary numbers

$0 + 0 = 0$
$0 + 1 = 1$
$1 + 0 = 1$
$1 + 1 = 0$ (carry one)

Addition:

$$\begin{array}{cccc} & 1 & 4 & 5 \\ + & & 2 & 3 \\ \hline & 1 & 6 & 8 \end{array}$$

$$\begin{array}{cccccccc} & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ + & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ \hline \end{array}$$

Arithmetic operations with binary numbers

$0 + 0 = 0$
$0 + 1 = 1$
$1 + 0 = 1$
$1 + 1 = 0$ (carry one)

Addition:

$$\begin{array}{cccc} & 1 & 4 & 5 \\ + & & 2 & 3 \\ \hline & 1 & 6 & 8 \end{array}$$

$$\begin{array}{cccccccc} & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ + & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ \hline & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \end{array}$$

Arithmetic operations with binary numbers

Addition:

$0+0=0$
$0+1=1$
$1+0=1$
$1+1=0$ (carry one)

$$\begin{array}{rrrr} & 1 & 4 & 5 \\ + & & 2 & 3 \\ \hline & 1 & 6 & 8 \end{array}$$

$$\begin{array}{rrrrrrrrr} & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ + & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ \hline & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \end{array}$$

Subtraction:

$0-0=0$
$1-0=1$
$1-1=0$
$0-1=1$ (borrow one)

Arithmetic operations with binary numbers

Addition:

$0+0=0$
$0+1=1$
$1+0=1$
$1+1=0$ (carry one)

$$\begin{array}{rrrr} & 1 & 4 & 5 \\ + & & 2 & 3 \\ \hline & 1 & 6 & 8 \end{array}$$

$$\begin{array}{rrrrrrrrr} & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ + & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ \hline & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \end{array}$$

Subtraction:

$0-0=0$
$1-0=1$
$1-1=0$
$0-1=1$ (borrow one)

$$\begin{array}{rrrr} & 1 & 4 & 5 \\ - & & 2 & 3 \\ \hline & 1 & 2 & 2 \end{array}$$

Arithmetic operations with binary numbers

$0+0=0$
$0+1=1$
$1+0=1$
$1+1=0$ (carry one)

Addition:

$$\begin{array}{rrrr} & 1 & 4 & 5 \\ + & & 2 & 3 \\ \hline & 1 & 6 & 8 \end{array}$$

$$\begin{array}{rrrrrrrrr} & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ + & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ \hline & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \end{array}$$

$0-0=0$
$1-0=1$
$1-1=0$
$0-1=1$ (borrow one)

Subtraction:

$$\begin{array}{rrrr} & 1 & 4 & 5 \\ - & & 2 & 3 \\ \hline & 1 & 2 & 2 \end{array}$$

$$\begin{array}{rrrrrrrrr} & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ - & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ \hline \end{array}$$

Arithmetic operations with binary numbers

Addition:

$0+0=0$
$0+1=1$
$1+0=1$
$1+1=0$ (carry one)

$$\begin{array}{rrrr} & 1 & 4 & 5 \\ + & & 2 & 3 \\ \hline & 1 & 6 & 8 \end{array} \qquad \begin{array}{rrrrrrrrr} & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ + & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ \hline & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \end{array}$$

Subtraction:

$0-0=0$
$1-0=1$
$1-1=0$
$0-1=1$ (borrow one)

$$\begin{array}{rrrr} & 1 & 4 & 5 \\ - & & 2 & 3 \\ \hline & 1 & 2 & 2 \end{array} \qquad \begin{array}{rrrrrrrrr} & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ - & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ \hline & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 0 \end{array}$$

- Multiplication and division are more expensive, and more elaborate

Loss of digits

- During operations such as $+, -, \times, \div$, an error can add up
- Consider the summation of x and y

$$\tilde{x} - \delta \leq x \leq \tilde{x} + \delta \quad \text{and} \quad \tilde{y} - \varepsilon \leq y \leq \tilde{y} + \varepsilon$$

$$(\tilde{x} + \tilde{y}) - (\delta + \varepsilon) \leq x + y \leq (\tilde{x} + \tilde{y}) + (\delta + \varepsilon)$$

- Consider the subtraction of x and y

$$(\tilde{x} - \tilde{y}) - (\delta + \varepsilon) \leq x - y \leq (\tilde{x} - \tilde{y}) + (\delta + \varepsilon)$$

Loss of digits: Example 1

$x = \pi, \tilde{x} = 3.1416$

$y = 22/7, \tilde{y} = 3.1429$

Loss of digits: Example 1

$$\left.\begin{array}{l} x = \pi, \tilde{x} = 3.1416 \\ y = 22/7, \tilde{y} = 3.1429 \end{array}\right\} \Rightarrow \left.\begin{array}{l} \delta = \tilde{x} - x = 7.35 \times 10^{-6} \\ \varepsilon = \tilde{y} - y = 4.29 \times 10^{-5} \end{array}\right\}$$

Loss of digits: Example 1

$$\left.\begin{array}{l} x = \pi, \tilde{x} = 3.1416 \\ y = 22/7, \tilde{y} = 3.1429 \end{array}\right\} \Rightarrow \left.\begin{array}{l} \delta = \tilde{x} - x = 7.35 \times 10^{-6} \\ \varepsilon = \tilde{y} - y = 4.29 \times 10^{-5} \end{array}\right\}$$

$$x + y = \tilde{x} + \tilde{y} \pm (\delta + \varepsilon) \approx 6.2845 - 5.025 \times 10^{-5}$$

$$x - y = \tilde{x} - \tilde{y} \pm (\delta + \varepsilon) \approx -0.0013 + 3.55 \times 10^{-5}$$

- The absolute error is small ($\approx 10^{-5}$), but the relative error is much bigger (0.028).
- Adding up the errors results in a loss of significant digits!

Catastrophic cancellation: concept

Catastrophic cancellation occurs when subtracting nearly equal numbers.
Relative error scales as

$$\frac{|\delta a - \delta b|}{|a - b|}.$$

When $|a - b|$ is small, relative error becomes large.

Loss of digits: Summation order matters

Example:

$$10^8 + \sum_{i=1}^{N} 10^{-8}.$$

```
N = 1000000
big = 1e8
tiny = 1e-8

s1 = big
for _ in range(N):
    s1 += tiny

s2 = 0.0
for _ in range(N):
    s2 += tiny
s2 += big

print(s1, s2)
```

Exercise: summation of many numbers

```python
import numpy as np
np.random.seed(42) # Make sure that the results are reproducible
x = 10.0**np.random.uniform(-12, 12, size=1_000_000)
```

- Sum the numbers in `x` in the given order.
- Now sort the numbers in ascending order and sum them again.
- Finally, sort the numbers in descending order and sum them again.
- What do you observe?

Exercise: summation of many numbers

```python
import numpy as np
np.random.seed(42) # Make sure that the results are reproducible
x = 10.0**np.random.uniform(-12, 12, size=1_000_000)
```

- Sum the numbers in `x` in the given order.
- Now sort the numbers in ascending order and sum them again.
- Finally, sort the numbers in descending order and sum them again.
- What do you observe?

Specialized algorithms (Kahan summation, pairwise summation) can reduce the error.

Loss of digits: Example 3

- Calculate e^{-5}
  - Use the Taylor series we programmed before ($e^x \approx \sum_{n=0}^{N} \frac{x^n}{n!}$)
  - Calculate the first 26 terms ($N = 26$)
- Now repeat the calculation, but use for each calculation only 4 digits. What do you find?

- Without errors you would find: $e^{-5} = 0.006738$
- If you only use 4 digits in the calculations, you'll find 0.00998

Loss of digits: Example 3

- Calculate e^{-5}
  - Use the Taylor series we programmed before ($e^x \approx \sum_{n=0}^{N} \frac{x^n}{n!}$)
  - Calculate the first 26 terms ($N = 26$)
- Now repeat the calculation, but use for each calculation only 4 digits. What do you find?
  Use: `round(term, 4)`
- Without errors you would find: $e^{-5} = 0.006738$
- If you only use 4 digits in the calculations, you'll find 0.00998

Loss of digits: Example 3

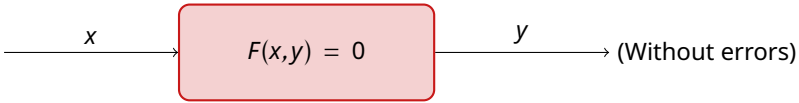
- Calculate e^{-5}
  - Use the Taylor series we programmed before ($e^x \approx \sum_{n=0}^{N} \frac{x^n}{n!}$)
  - Calculate the first 26 terms ($N = 26$)
- Now repeat the calculation, but use for each calculation only 4 digits. What do you find?
  Use: `round(term, 4)`
- Without errors you would find: $e^{-5} = 0.006738$
- If you only use 4 digits in the calculations, you'll find 0.00998

Loss of digits: Example 3

- Calculate e^{-5}
  - Use the Taylor series we programmed before ($e^x \approx \sum_{n=0}^{N} \frac{x^n}{n!}$)
  - Calculate the first 26 terms ($N = 26$)
- Now repeat the calculation, but use for each calculation only 4 digits. What do you find?
  Use: `round(term, 4)`
- Without errors you would find: $e^{-5} = 0.006738$
- If you only use 4 digits in the calculations, you'll find 0.00998

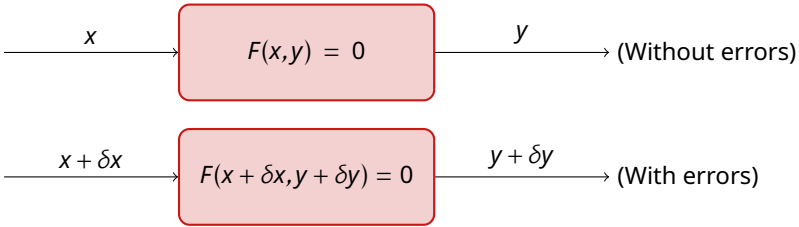
Badly (ill) conditioned problems

We consider a system $F(x,y)$ that computes a solution from input data. The input data may have errors:



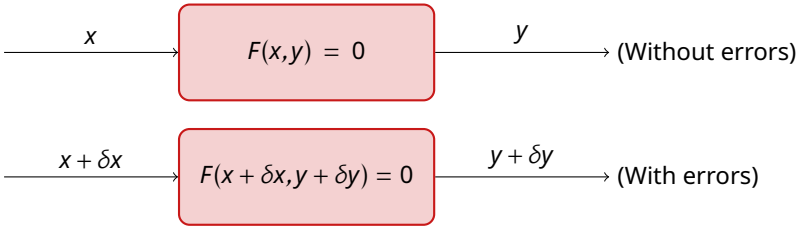
Badly (ill) conditioned problems

We consider a system $F(x,y)$ that computes a solution from input data. The input data may have errors:



Badly (ill) conditioned problems

We consider a system $F(x,y)$ that computes a solution from input data. The input data may have errors:



$$y(x+\delta x) - y(x) \approx y'(x)\delta x$$

Propagated error on the basis of Taylor expansion

$$C = \max_{\delta x}\left(\left|\frac{\delta y/y}{\delta x/x}\right|\right)$$

Condition criterion, $C < 10$ error development small

Badly (ill) conditioned problems: Example

Solve the following linear system in Python using double and single precision:

$$A = \begin{bmatrix} 1.2969 & 0.8648 \\ 0.2161 & 0.1441 \end{bmatrix}, \quad x = \begin{bmatrix} 0.8642 \\ 0.1440 \end{bmatrix}$$

Badly (ill) conditioned problems: Example

Solve the following linear system in Python using double and single precision:

$$A = \begin{bmatrix} 1.2969 & 0.8648 \\ 0.2161 & 0.1441 \end{bmatrix}, \quad x = \begin{bmatrix} 0.8642 \\ 0.1440 \end{bmatrix}, \quad y = \begin{bmatrix} 2.0 \\ -2.0 \end{bmatrix}$$

Badly (ill) conditioned problems: Example

Solve the following linear system in Python using double and single precision:

$$A = \begin{bmatrix} 1.2969 & 0.8648 \\ 0.2161 & 0.1441 \end{bmatrix}, \quad x = \begin{bmatrix} 0.8642 \\ 0.1440 \end{bmatrix}, \quad y = \begin{bmatrix} 2.0 \\ -2.0 \end{bmatrix}$$

Double precision

```
1 import numpy as np
2
3 # Double precision
4 A = np.array([[1.2969, 0.8648], [0.2161,
      0.1441]], dtype=np.float64)
5 x = np.array([0.8642, 0.1440], dtype=np.float64)
6 y = np.linalg.solve(A, x)
7 print("y =")
8 print(y)
```

Badly (ill) conditioned problems: Example

Solve the following linear system in Python using double and single precision:

$$A = \begin{bmatrix} 1.2969 & 0.8648 \\ 0.2161 & 0.1441 \end{bmatrix}, \quad x = \begin{bmatrix} 0.8642 \\ 0.1440 \end{bmatrix}, \quad y = \begin{bmatrix} 2.0 \\ -2.0 \end{bmatrix}$$

Double precision

```
1 import numpy as np
2
3 # Double precision
4 A = np.array([[1.2969, 0.8648], [0.2161,
      0.1441]], dtype=np.float64)
5 x = np.array([0.8642, 0.1440], dtype=np.float64)
6 y = np.linalg.solve(A, x)
7 print("y =")
8 print(y)
```

Single precision

```
1 import numpy as np
2
3 # Single precision
4 A = np.array([[1.2969, 0.8648], [0.2161,
      0.1441]], dtype=np.float32)
5 x = np.array([0.8642, 0.1440], dtype=np.float32)
6 y = np.linalg.solve(A, x)
7 print("y =")
8 print(y)
```

Badly (ill) conditioned problems: Example

- Python does not automatically warn about bad condition number. You need to compute and check it manually using NumPy:

```
if np.linalg.cond(A)> threshold: raise("Ill conditioned error")
```

- The `cond` number is the condition number computed using NumPy.
- A small error in A (the matrix) results in a big error in the solution. This is called an ill-conditioned problem.

Badly (ill) conditioned problems: Example

- Python does not automatically warn about bad condition number. You need to compute and check it manually using NumPy:

```
if np.linalg.cond(A)> threshold: raise("Ill conditioned error")
```

- The `cond` number is the condition number computed using NumPy.
- A small error in A (the matrix) results in a big error in the solution. This is called an ill-conditioned problem.

Badly (ill) conditioned problems: Example

- Python does not automatically warn about bad condition number. You need to compute and check it manually using NumPy:

```
if np.linalg.cond(A)> threshold: raise("Ill conditioned error")
```

- The `cond` number is the condition number computed using NumPy.
- A small error in A (the matrix) results in a big error in the solution. This is called an ill-conditioned problem.

Today's outline

- Introduction
- Roundoff errors
- Break errors
- Loss of digits
- (Un)stable methods
  - Error propagation
- Symbolic math
- Summary

(Un)stable methods

- The condition criterion does not tell you anything about the quality of a numerical solution method!
- It is very well possible that a certain solution method is more sensitive for one problem than another
- If the method propagates the error, we call it an *unstable method*. Let's look at an example.

Error propagation: concept

Errors propagate through numerical algorithms.
For an iteration

$$x_{n+1} = g(x_n),$$

the error satisfies

$$e_{n+1} \approx g'(x^*)e_n + \delta_n.$$

If $|g'(x^*)| < 1$, errors decay. If $|g'(x^*)| > 1$, errors amplify.

Error propagation: numerical differentiation

Forward difference:

$$f'(x) \approx \frac{f(x+h)-f(x)}{h}.$$

Truncation error is $O(h)$, roundoff error behaves like $O(\varepsilon/h)$.
Example: $f(x) = \sin x$ at $x = 1$.

```
import numpy as np

x = 1.0
true = np.cos(x)

for k in range(1,17):
    h = 10.0**(-k)
    approx = (np.sin(x+h)-np.sin(x))/h
    print(h, abs(approx-true))
```

Error propagation: mitigation

- Use higher-order schemes (e.g. central differences).
- Choose h where truncation and roundoff balance.
- Avoid subtraction when possible (symbolic or automatic differentiation).

Smaller step sizes are not always better.

Example 1: Explanation

Recall example 1, where the errors blew up our computation of 0.1, whereas they did not for 2. Why did we see these results?

Example 1: Explanation

Recall example 1, where the errors blew up our computation of 0.1, whereas they did not for 2. Why did we see these results?

- The number 0.1 is not exactly represented in binary
  - A tiny error can accumulate up to catastrophic proportions!
- The number 2 does have an exact binary representation

Example 1: Explanation

Recall example 1, where the errors blew up our computation of 0.1, whereas they did not for 2. Why did we see these results?

- The number 0.1 is not exactly represented in binary
  - A tiny error can accumulate up to catastrophic proportions!
- The number 2 does have an exact binary representation

Example 2 (large sine series)

The `np.sin(1e40*np.pi)` result gives poor results because 1e40 has an error margin on the order of floating-point machine epsilon, which is roughly 1×10^{-16} in Python (double-precision floating-point format). In Python, as in many computing environments, the number of $2 \cdot \pi$ cycles is still much larger than $10^{40} \cdot 10^{-16}$. Also, π is not stored with an infinite number of digits, which further contributes to the imprecision.

Example 3

Start your calculation program of choice (Excel, Python, ...)

Example 3

Start your calculation program of choice (Excel, Python, ...)

Calculate the result of y:

$$y = e^{\pi} - \pi$$

Example 3

Start your calculation program of choice (Excel, Python, ...)

Calculate the result of y:

$$y = e^{\pi} - \pi = 19.999099979$$

Example 3

Start your calculation program of choice (Excel, Python, ...)

Calculate the result of y:

$$y = e^{\pi} - \pi = 19.999099979 \neq 20$$

Example 3

Start your calculation program of choice (Excel, Python, ...)

Calculate the result of y:

$$y = e^{\pi} - \pi = 19.999099979 \neq 20$$

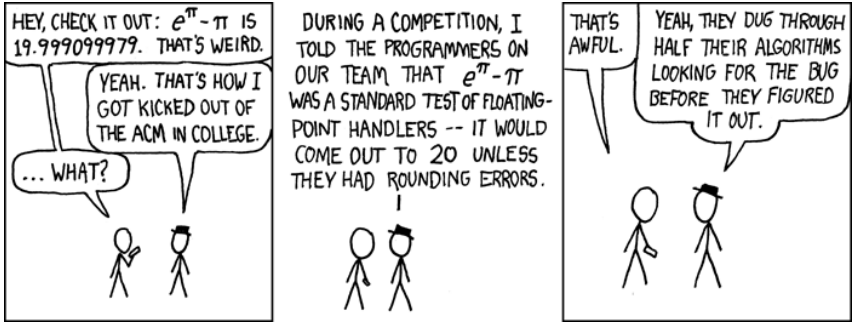


Image: xkcd

Today's outline

- Introduction
- Roundoff errors
- Break errors
- Loss of digits
- (Un)stable methods
  - Error propagation
- Symbolic math
- Summary

Symbolic math packages

Definition

The use of computers to manipulate mathematical equations and expressions in symbolic form, as opposed to manipulating the numerical quantities represented by those symbols.

- Symbolic integration or differentiation, substitution of one expression into another
- Simplification of an expression, change of subject etc.
- Packages and toolboxes:

Symbolic math packages

Mathematica	Well known software package, license available via TU/e
Maple	Well known, license available via TU/e
Wolfram|Alpha	Web-based interface by Mathematica developer. Less powerful in mathematical respect, but more accessible and has a broad application range (unit conversion, semantic commands).
Sage	Open-source alternative to Maple, Mathematica, Magma, and MATLAB.
Matlab	Symbolic math toolbox
Python	SymPy

Symbolic math: simplify

$$f(x) = (x-1)(x+1)(x^2+1)+1$$

Symbolic math: simplify

$$f(x) = (x-1)(x+1)(x^2+1)+1$$

```python
from sympy import symbols, simplify

x = symbols('x') # Alt: from sympy.abc import x,y,z
f = (x - 1)*(x + 1)*(x**2 + 1) + 1
f_simplified = simplify(f)
print(f_simplified)
```

Symbolic math: simplify

$$f(x) = (x-1)(x+1)(x^2+1)+1$$

```
from sympy import symbols, simplify

x = symbols('x') # Alt: from sympy.abc import x,y,z
f = (x - 1)*(x + 1)*(x**2 + 1) + 1
f_simplified = simplify(f)
print(f_simplified)
```

```
f_simplified = x**4
```

Symbolic math: integration and differentiation

$$f(x) = \frac{1}{x^3 + 1}$$

Symbolic math: integration and differentiation

$$f(x) = \frac{1}{x^3 + 1}$$

```
from sympy import symbols, simplify, integrate, diff

x = symbols('x')
f = 1/(x**3+1)
my_f_int = integrate(f, x)
my_f_diff = diff(my_f_int, x)
my_f_diff_simplified = simplify(my_f_diff)
print(my_f_diff_simplified)
```

Symbolic math: integration and differentiation

$$f(x) = \frac{1}{x^3 + 1}$$

```
from sympy import symbols, simplify, integrate, diff

x = symbols('x')
f = 1/(x**3+1)
my_f_int = integrate(f, x)
my_f_diff = diff(my_f_int, x)
my_f_diff_simplified = simplify(my_f_diff)
print(my_f_diff_simplified)
```

```
my_f_diff_simplified = 1/(x**3 + 1)
```

Symbolic math: exercises

Exercise 1

Simplify the following expression:

$$f(x) = \frac{2\tan x}{(1+\tan^2 x)}$$

Symbolic math: exercises

Exercise 1

Simplify the following expression:

$$f(x) = \frac{2\tan x}{(1+\tan^2 x)} = \sin 2x$$

```
1 from sympy import symbols, trigsimp, tan
2
3 x = symbols('x')
4 expr = 2*tan(x)/(1 + tan(x)**2)
5 simplified_expr = trigsimp(expr)
```

Symbolic math: exercises

Exercise 2

Calculate the *value* of p:

$$p = \int_0^{10} \frac{e^x - e^{-x}}{\sinh x} dx$$

Symbolic math: exercises

Exercise 2

Calculate the *value* of p:

$$p = \int_0^{10} \frac{e^x - e^{-x}}{\sinh x} dx$$

```
from sympy import exp, sinh, integrate, symbols

x = symbols("x")
f = ((exp(x)- exp(-x))/sinh(x)).simplify()
p = integrate(f, (x, 0, 10))
print(p)
```

Symbolic math: exercises

Exercise 2

Calculate the *value* of p:

$$p = \int_0^{10} \frac{e^x - e^{-x}}{\sinh x} dx$$

```
from sympy import exp, sinh, integrate, symbols

x = symbols("x")
f = ((exp(x)- exp(-x))/sinh(x)).simplify()
p = integrate(f, (x, 0, 10))
print(p)
```

```
p = 20
```

Symbolic math: root finding

A root finding method searches for the values where a function reaches zero. We will cover the numerical methods later, here we show how to use root finding with symbolic math in Python.

Symbolic math function

$$f(x) = \frac{3}{x^2 + 3x} - 2$$

Symbolic math: root finding

A root finding method searches for the values where a function reaches zero. We will cover the numerical methods later, here we show how to use root finding with symbolic math in Python.

Symbolic math function

$$f(x) = \frac{3}{x^2 + 3x} - 2$$

```
from sympy import solve, symbols

x = symbols("x")
f = 3 / (x**2 + 3*x) - 2
solutions = solve(f, x)
print(solutions)
```

Symbolic math: root finding

A root finding method searches for the values where a function reaches zero. We will cover the numerical methods later, here we show how to use root finding with symbolic math in Python.

Symbolic math function

$$f(x) = \frac{3}{x^2 + 3x} - 2$$

```
from sympy import solve, symbols

x = symbols("x")
f = 3 / (x**2 + 3*x) - 2
solutions = solve(f, x)
print(solutions)
```

```
solutions = [-3/2 + sqrt(15)/2, -sqrt(15)/2 - 3/2]
```

Today's outline

- Introduction
- Roundoff errors
- Break errors
- Loss of digits
- (Un)stable methods
  - Error propagation
- Symbolic math
- Summary

Summary

- Numerical errors may arise due to truncation, roundoff and break errors, which may seriously affect the accuracy of your solution
- Errors may propagate and accumulate, leading to smaller accuracy
- Ill-conditioned problems and unstable methods have to be identified so that proper measures can be taken
- Symbolic math computations may be performed to solve certain equations algebraically, bypassing numerical errors, but this is not always possible.

Summary

- Numerical errors may arise due to truncation, roundoff and break errors, which may seriously affect the accuracy of your solution
- Errors may propagate and accumulate, leading to smaller accuracy
- Ill-conditioned problems and unstable methods have to be identified so that proper measures can be taken
- Symbolic math computations may be performed to solve certain equations algebraically, bypassing numerical errors, but this is not always possible.

Summary

- Numerical errors may arise due to truncation, roundoff and break errors, which may seriously affect the accuracy of your solution
- Errors may propagate and accumulate, leading to smaller accuracy
- Ill-conditioned problems and unstable methods have to be identified so that proper measures can be taken
- Symbolic math computations may be performed to solve certain equations algebraically, bypassing numerical errors, but this is not always possible.

Summary

- Numerical errors may arise due to truncation, roundoff and break errors, which may seriously affect the accuracy of your solution
- Errors may propagate and accumulate, leading to smaller accuracy
- Ill-conditioned problems and unstable methods have to be identified so that proper measures can be taken
- Symbolic math computations may be performed to solve certain equations algebraically, bypassing numerical errors, but this is not always possible.